

2.Study of simple linear regression model

Objective

To study and implement linear regression using Python libraries (NumPy, Pandas, Matplotlib, scikit-learn) for predicting student scores based on study hours.

Theory

Linear regression is a supervised machine learning algorithm used to model the relationship between an independent variable (X: study hours) and a dependent variable (Y: student scores). It fits a straight line $Y = b_0 + b_1X$, where b_0 is the intercept and b_1 is the slope. The model minimizes error between predicted and actual values, evaluated using metrics like Mean Squared Error (MSE) and R^2 score.

CODE;-

```
# ----- Import Libraries -----
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# ----- Step 1: Load Dataset -----
# Use raw string (r"....") to avoid Unicode errors in Windows
df = pd.read_csv(r"C:\Users\rajri\Documents\student_scores.csv")
print("Dataset shape:", df.shape)
print(df.head())

# Independent variable (Hours of study)
X = df[["Hours"]] # must be 2D
# Dependent variable (Scores)
y = df["Scores"]

# ----- Step 2: Split Data -----
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ----- Step 3: Train Model -----
```

```
model = LinearRegression()
model.fit(X_train, y_train)

# ----- Step 4: Predict -----
y_pred = model.predict(X_test)

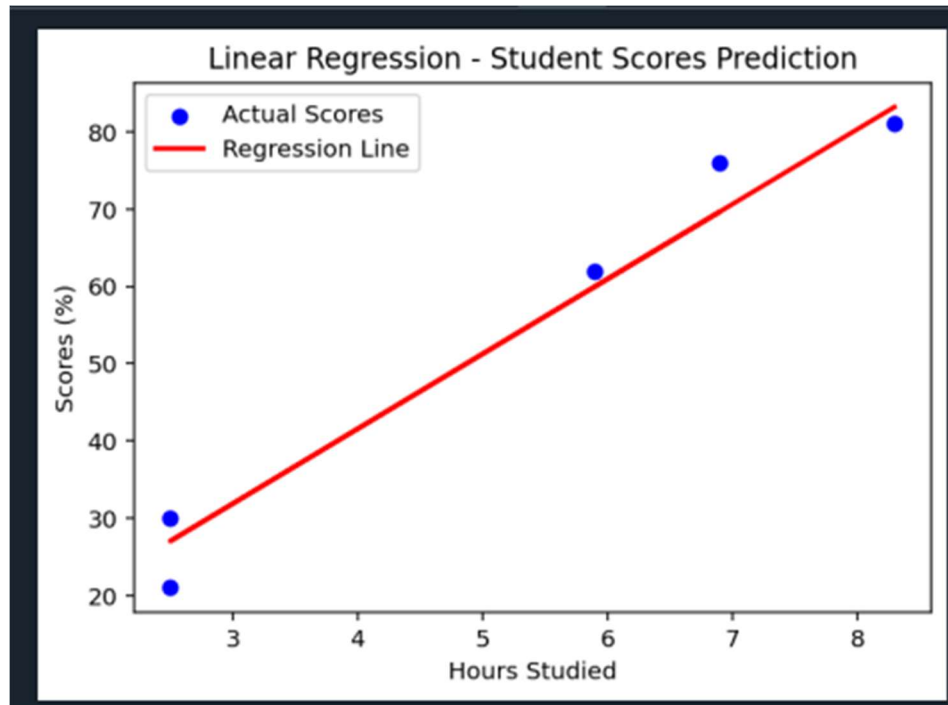
# ----- Step 5: Evaluate Model -----
print("\n--- Model Parameters ---")
print("Intercept (b0):", model.intercept_)
print("Coefficient (b1):", model.coef_[0])

print("\n--- Performance ---")
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))

# ----- Step 6: Visualization -----
plt.scatter(X_test, y_test, color="blue", label="Actual Scores")
plt.plot(X_test, y_pred, color="red", linewidth=2, label="Regression Line")
plt.title("Linear Regression - Student Scores Prediction")
plt.xlabel("Hours Studied")
plt.ylabel("Scores (%)")
plt.legend()
plt.show()

# ----- Step 7: User Input Prediction -----
try:
    hours = float(input("\nEnter number of study hours: "))
    predicted_score = model.predict([[hours]]) # 2D input required
    print(f"Predicted Score for {hours} hours of study = {predicted_score[0]:.2f}")
except ValueError:
    print("Invalid input! Please enter a numeric value for hours.")
```

OUTPUT:-



Dataset shape: (25, 2)

Hours Scores

0	2.5	21
1	5.1	47
2	3.2	27
3	8.5	75
4	3.5	30

--- Model Parameters ---

Intercept (b0): 2.826892353899737

Coefficient (b1): 9.682078154455697

--- Performance ---

Mean Squared Error: 18.943211722315272

R2 Score: 0.9678055545167994

Enter number of study hours: 10

Predicted Score for 10.0 hours of study = 99.65